

5) Document Ingestion

Last updated by | Mennell, Jacob | 19 May 2026 at 12:25 BST

Document Ingestion

Purpose

This page describes how source documents are transformed into retrieval-ready chunks that can later be searched, reranked, and used by AI-assisted extraction and checking workflows.

Scope

The ingestion flow shown in the repository is responsible for:

- extracting document text and structure
- reconstructing a normalised textual representation
- splitting content into chunks suited for retrieval
- enriching chunks with metadata such as header hierarchy and layout information
- generating embeddings for those chunks
- persisting the resulting records into the database

High-Level Ingestion Flow

The ingestion module documents a multi-step flow built around Azure Document Intelligence and a fallback path.

At a high level, the process is:

1. analyze the source document with Azure Document Intelligence
2. rebuild a normalized markdown-like representation of the document
3. infer heading structure and maintain header hierarchy
4. split the content into retrieval-oriented chunks
5. rebalance chunks for token budgets and special content types such as tables
6. generate embeddings for each chunk
7. store chunk text, embeddings, and metadata in the database

Source Parsing And Structure Recovery

The code indicates that the preferred ingestion path uses a single Document Intelligence layout call. The resulting content is used to reconstruct a markdown-like document that preserves structure in a form suitable for later semantic splitting.

This reconstructed representation is then normalized to reduce noise. The normalization logic includes:

- newline normalization
- collapsing excessive blank lines
- removal of consecutive duplicate headings

This step matters because chunk quality depends on preserving structural cues while removing formatting artifacts that do not help retrieval.

Header-Aware Chunking

The ingestion logic does not treat chunking as simple fixed-size splitting. Instead, it keeps track of header hierarchy and produces a flattened `header_path` such as `Section > Subsection > Detail`.

The resulting chunk metadata can include:

- stable chunk hash
- semantic source identifier
- header map and flattened header path
- section identifier
- page number
- page-level and part-level metadata from Document Intelligence
- optional geometry and layout block information

This approach is important because downstream retrieval can use structural cues such as header paths in addition to the raw chunk body.

Table-Aware Rebalancing

The repository explicitly treats markdown-style tables as a special case during chunk rebalancing. Oversized tables are split by row while repeating the table header, and nearby rows may be carried forward into the next chunk as bounded overlap when that still fits within the token budget.

This is a meaningful design choice because table content often loses its semantics when naively split at arbitrary token boundaries.

For non-table text, the system uses overlap-aware chunk splitting and can merge undersized adjacent chunks when they remain within the same structural context.

Layout And Geometry Enrichment

The ingestion code attempts to map chunks back to layout blocks and page geometry on a best-effort basis. When successful, the stored metadata can include:

- page-level provenance
- matched layout blocks
- chunk bounding box information

This is useful for traceability and can support later experiences that require chunk-to-document or chunk-to-page attribution.

Embedding Generation

After structured chunks are produced, the ingestion flow embeds chunk text in batches using the configured embedding model. The batching logic is explicit and configurable through the embedding batch size.

This embedding stage turns each chunk into a retrieval-ready vector representation that can later be used by semantic and hybrid retrieval flows.

Persistence Model

Chunk persistence stores, at minimum:

- document_id
- claim_id
- chunk_index
- text_chunk
- document_type
- embedding
- meta_data
- optional chunk_context

This data is stored in the document chunk table backed by PostgreSQL and pgvector.

Fallback Behavior

The ingestion module documents a fallback path used when the Document Intelligence flow fails or is intentionally bypassed. The fallback produces usable chunks but with reduced structural richness. In particular, header paths and layout-derived metadata may be absent or minimal.

This means the solution is designed to fail soft on parsing rather than block downstream processing entirely.

Design Decisions

- Structure preservation is treated as part of retrieval quality, not just parsing quality.
- Table handling is explicit rather than left to generic text splitting.
- Chunk metadata is rich enough to support later retrieval, attribution, and debugging.
- Embedding is part of ingestion, so retrieval does not have to compute document vectors at query time.
- Fallback ingestion preserves system continuity when the preferred parsing path is unavailable.

Operational Notes

- Ingestion quality directly affects downstream retrieval and AI workflow quality.
- Documents with weak structural signals may still be ingested, but retrieval behavior may rely more heavily on chunk text than on hierarchy metadata.
- Metadata richness is higher on the Document Intelligence path than on the fallback path.